

UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Course Name: Operating System Lab

Course Code: CSE 402

LAB REPORT

Lab Report Number: 08

Lab Report Name: Implementation of the FIFO (First In First Out) Page Replacement Algorithm

Submitted By: **Name:** Tashnimul Hasan Shifat **Reg:** 23101142 **Section:** C2	**Submitted To:** Atia Rahman Orthi Lecturer University of Asia Pacific

Date of Submission: 15.10.2026

1. Problem Statement:

Main memory is limited, so the operating system cannot keep every page of a running process in physical memory at the same time. When a process references a page that is not currently loaded in any frame, a page fault occurs and the required page must be brought in from secondary storage. If all frames are already occupied, the operating system must choose an existing page to remove. The rule used to make this choice is called a page replacement algorithm, and its quality directly affects how often costly page faults occur.

The objective of this laboratory task is to implement the First In First Out (FIFO) page replacement algorithm in Python. FIFO treats the frames as a queue: whenever a replacement is required, the page that was loaded into memory earliest is evicted. Given the reference string 1, 2, 3, 4, 2, 3, 1 and a frame size of 3, the program must classify every reference as a page hit or a page fault, display the frame contents after each reference, and finally compute the total number of hits, total number of faults, hit ratio and fault ratio.

2. Objectives:

- To understand the concept of paging, page faults and the need for page replacement.
- To understand the working principle of the FIFO page replacement algorithm.
- To implement the FIFO algorithm in Python using a list as a queue of frames.
- To trace the frame contents after each page reference and identify hits and faults.
- To calculate the total hits, total faults, hit ratio and fault ratio for the given input.
- To analyze the advantages and limitations of the FIFO page replacement policy.

3. Source Code Screenshot:

```python
# Page Replacement Algorithm (FIFO)

reference = [1, 2, 3, 4, 2, 3, 1]
frame_size = 3

frame = []
hit = 0
fault = 0

for page in reference:

    if page in frame:
        hit += 1
        print(page, "->", frame, "Hit")

    else:
        fault += 1

        if len(frame) < frame_size:
            frame.append(page)
        else:
            frame.pop(0)
            frame.append(page)
```

Figure 1: Initialization and FIFO page replacement logic

```python
            print(page, "->", frame, "Fault")

hit_ratio = (hit / len(reference)) * 100
fault_ratio = (fault / len(reference)) * 100

print("Total Hit =", hit)
print("Total Fault =", fault)
print("Hit Ratio =", round(hit_ratio,2),"%")
print("Fault Ratio =", round(fault_ratio, 2),"%")
```

Figure 2: Calculation and display of hit and fault ratios

4. Output:

```
1 -> [1] Fault
2 -> [1, 2] Fault
3 -> [1, 2, 3] Fault
4 -> [2, 3, 4] Fault
2 -> [2, 3, 4] Hit
3 -> [2, 3, 4] Hit
1 -> [3, 4, 1] Fault
Total Hit = 2
Total Fault = 5
Hit Ratio = 28.57 %
Fault Ratio = 71.43 %

[Done] exited with code=0 in 0.105 seconds
```

Figure 3: Execution output of the FIFO page replacement program

5. Discussion:

The implemented program simulates the FIFO page replacement algorithm using a Python list to represent the memory frames. For every page in the reference string, the program first checks whether the page is already present in the frames. If it is, the reference is counted as a hit and the frames remain unchanged. Otherwise, the reference is counted as a fault; the page is appended directly if a free frame is available, or, if all frames are full, the oldest page at the front of the list is removed using pop(0) before the new page is appended at the end. The step-by-step behaviour for the given input is summarized in Table 1.

Table 1: Frame status after each page reference (Frame size = 3)

Step	Page	Frame 1	Frame 2	Frame 3	Page Replaced	Status
1	1	1	–	–	–	Fault
2	2	1	2	–	–	Fault
3	3	1	2	3	–	Fault
4	4	2	3	4	1	Fault
5	2	2	3	4	–	**Hit**
6	3	2	3	4	–	**Hit**
7	1	3	4	1	2	Fault

The following observations were made from the experiment:

- The first three references (pages 1, 2 and 3) are compulsory faults, because the frames are initially empty and every page has to be loaded for the first time.

- When page 4 arrives, all three frames are occupied. Page 1 entered memory first, so it is selected as the victim and the frames become [2, 3, 4].
- The next references to pages 2 and 3 are hits, since both pages are still present in memory. A hit does not change the order of the queue in FIFO.
- When page 1 is referenced again, it is no longer in memory and causes a fault. Page 2 is now the oldest page in the frames, so it is replaced and the frames become [3, 4, 1].
- Overall, the program recorded 2 hits and 5 faults out of 7 references, giving a hit ratio of 28.57% and a fault ratio of 71.43%.
- The page 1 fault at the end illustrates a key weakness of FIFO: page 1 was removed only because it was the oldest, not because it was unlikely to be needed again. FIFO ignores how recently or how frequently a page is used.
- FIFO is simple to implement and has very low overhead, since it only needs to remember the arrival order of pages. However, it may suffer from Belady's Anomaly, where increasing the number of frames can sometimes increase the number of page faults.
- With the list-based implementation used here, the membership check and pop(0) operation each take O(f) time, where f is the frame size, giving an overall time complexity of O(n × f) for n page references.

6. Conclusion:

In this experiment, the FIFO page replacement algorithm was successfully implemented in Python and tested using the reference string 1, 2, 3, 4, 2, 3, 1 with a frame size of 3. The program correctly displayed the frame contents after every reference and produced 2 page hits and 5 page faults, resulting in a hit ratio of 28.57% and a fault ratio of 71.43%.

It can be concluded that FIFO is easy to understand and implement because it replaces pages strictly in the order in which they were loaded. However, since it does not take page usage history into account, it may evict pages that are still needed, which leads to additional page faults. Algorithms such as Optimal or Least Recently Used (LRU) can achieve better performance by considering future or past page usage. The experiment provided practical understanding of page faults, page hits and the behaviour of the FIFO replacement policy.

7. Git Link:

https://github.com/shifat35/Operation_System_Lab-CSE-402.git